

FINAL REPORT: TRAFFIC SIGN DETECTION AND CLASSIFICATION WITH DEEP LEARNING

Christopher Lee

Student# 1011048723

christ.lee@mail.utoronto.ca

Ryan Hammer

Student# 1011055144

ryan.hammer@mail.utoronto.ca

Justin Fang

Student# 1010143881

justin.fang@mail.utoronto.ca

Willis Wu

Student# 1010955110

willis.wu@mail.utoronto.ca

ABSTRACT

This project is a prototype system for traffic-sign detection and classification aimed at improving road-scene understanding for autonomous vehicles. Focusing on North American signs, the team will leverage the Mapillary Traffic Sign Dataset (MTSD) to train a two-stage pipeline. In Stage 1, non-deep learning classical computer-vision techniques (color thresholding, Canny edge detection, and contour filtering in OpenCV) generate bounding-box proposals for candidate signs. In Stage 2, a Convolutional Neural Network classifies each cropped region into its specific sign category. The project plan details dataset preprocessing, model architecture, baseline OpenCV comparisons, ethical considerations, and risk mitigation. Successful completion will yield a Colab notebook capable of detecting and labeling traffic signs in diverse conditions. —Total Pages: 8

1 INTRODUCTION

Recognizing traffic signs is an essential task for autonomous vehicles. Without it, a car could miss a speed limit change or fail to stop, creating dangerous situations. Because of this, accurate traffic sign detection and classification is a growing point of research in both industry and academia.

In our project, we built a prototype that handles detection and classification in two stages. Stage one uses transfer learning on a pretrained object detection model to identify multiple signs. Stage two takes the detected regions and classifies them using a Convolutional Neural Network (CNN).

The motivation for this approach comes from the fact that signs in the real world rarely appear in perfect conditions, making rule-based methods unreliable. Machine learning can adapt to these variations by recognizing subtle visual patterns, similar to a human driver. Our goal was to create a working prototype that showcases the potential of combining deep learning detection with classification, providing a solid foundation for an automated traffic sign recognition system.

2 ILLUSTRATION / FIGURE

2 Stage Detection / Classification System

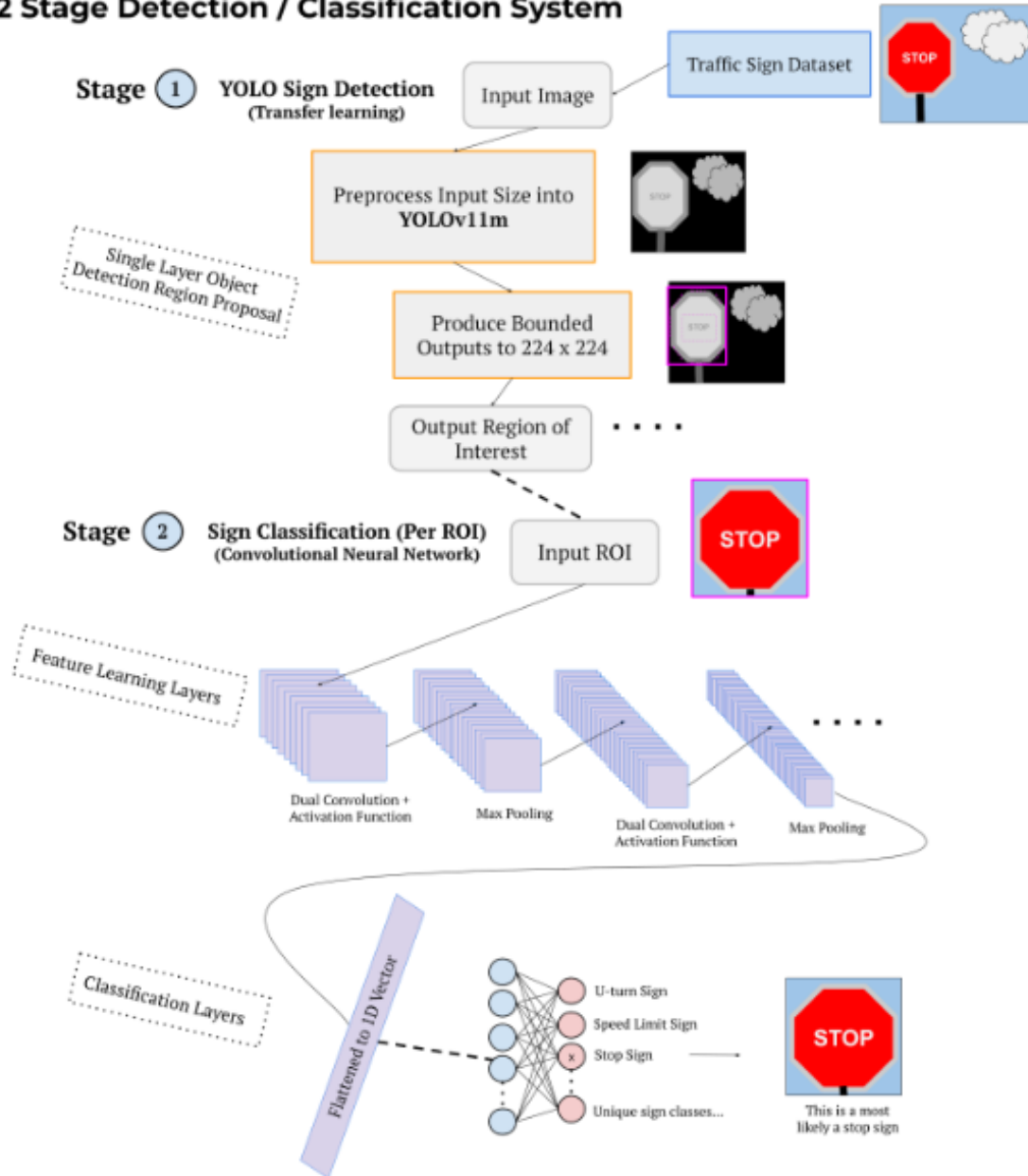


Figure 1: Overview of Two-Stage Traffic Sign Detection and Classification Pipeline

3 BACKGROUND AND RELATED WORK

Traffic sign recognition (TSR) is a critical component of autonomous driving systems, allowing vehicles to interpret their surroundings and make informed decisions. This section introduces some related work in the field to show what has been done in this area.

The Multi-column Deep Neural Network (MCDNN) was one of the first machine learning models for traffic sign recognition, trained on the GTSRB benchmark dataset ([Mykola, 2018](#)). It combined multiple independently trained CNNs with different initializations and augmentations, using three convolutional layers for feature extraction and two fully connected layers for classification. This approach achieved high accuracy and even surpassed human performance ([Cireşan, 2012](#)).

Google Street View systems demonstrate applications in diverse real-world geographic environments by using object detection combined with optical character recognition (extracting text from images) to recognize street signs and text under various conditions such as occlusion, glare, and deformation ([Maps, 2025](#)).

YOLOv8 is a real-time object detection model named "You Only Look Once." Its speed and accuracy make it suitable for embedded vehicle systems with limited computing power. These models detect objects by dividing an image into a grid and predicting bounding boxes and class probabilities for each grid cell in a single pass. This allows for fast and real-time detection of traffic signs ([Ultralytics, 2023](#)).

Although computationally more demanding, vision transformers (ViTs) are capable of detecting traffic signs even in challenging conditions, such as heavy snow, rain, and dark environments. ViTs splits an image into patches and processes them like words in a sentence using self-attention mechanisms. This allows the model to capture relationships between distant parts of the image, improving recognition in complex or cluttered scenes ([Mingwin, 2024](#)).

Real-time embedded deployment focuses on running AI models directly on small, power-efficient hardware that can process inputs and produce outputs instantly ([Tsai, 2017](#)). The NVIDIA Jetson Nano is one example, as a compact, low-power board designed to run AI models on-device. Its small size, energy efficiency, and low latency also make it well suited for vehicle use, showing that accurate, real-time traffic sign recognition is possible in a compact form factor ([Artamonov & Yakimov, 2018](#)).

Together, these works lay the foundation for our project, which aims to combine diverse traffic sign data and explore CNN-based architectures for effective traffic sign recognition.

4 DATA PROCESSING

The dataset used for both stages of the model was pulled from the Mapillary Traffic Sign Dataset ([Ertler, 2019](#)). While preliminary research indicated that the dataset contained 312 classes, a verification script parsing through all annotations revealed that there were, in fact, 401 distinct classes. The stage one dataset consisted of 41,916 images split into a ratio of 70/15/15 for training, validation, and testing.

Data processing was the largest obstacle the team faced, as the MTSD dataset provided little documentation on use cases, and images varied widely in content and quality (size). The variation in image sizes made it challenging to choose a resizing method that preserved as much visual detail as possible. Our group initially resized the images to preserve aspect ratio, however, it required separate scaling factors for width and height, along with precise padding and offset adjustments for every bounding box. This made the process significantly more complex and error-prone, leading us to scrap the idea.

A major problem arose from the fact that annotations were not paired directly to their corresponding images, meaning that each .json annotation had to be connected to every image. The team used a custom class to read each annotation in .json form, pull critical information (bounding boxes, class type output, and the

image key, and link it to its corresponding image for training. The dataset’s large size introduced additional technical obstacles, as the team encountered operating system “Input/output error” exceptions every time the Colab was run, forcing us to restart the runtime multiple times. To address this, we developed a custom class designed to individually load image annotation pairs rather than all at once, preventing RAM overload.

After successful pairing, we normalized and resized all images to 1024x1024 pixels, with the resolution chosen as it was the optimal balance between visual consistency and quality, given original image sizes ranged from 720x1020 pixels to 4k. Resizing the samples impacted small and distant traffic signs, as detail was lost when reducing dimensionality. Following this, all bounding box coordinates were resized for each image using a custom script, with this particular step leading to many errors. Incorrect scaling calculations initially produced misaligned bounding boxes, causing us to restart each time. Processing time also represented a lot of obstacles, as resizing all 41k files took 14 hours, with multiple runtime disconnections.

For the second stage, we built a new dataset from scratch using the original MTSD images. This process involved reviewing each image, isolating every traffic sign using annotations, cropping it, and generating a new annotation for each resulting sub-image. Our initial attempt to store annotations in a .txt file failed to save correctly, but when the group pivoted to .csv files, it introduced further issues with saving the new annotations. Cropping and resizing of areas of interest worked, but missing or incomplete annotations forced us to repeat portions of the workflow multiple times. Due to processing time and dataset size, the team only processed 26,000 of the original images, yielding over 130k individual sign images, split into training, validation, and testing, 80/10/10 ratio. We also encountered orphan images without annotations, but ultimately, images were normalized and resized to 224x224 pixels ([Rahman, 2025](#)).



Figure 2: Original image pre-processing; next to it is the same image resized and normalized.

5 ARCHITECTURE

The deep-learning system the team developed is a two-stage traffic sign detection and classification pipeline. Stage one uses a pretrained YOLOv11m object detection model via transfer learning to localize traffic signs in input images, while stage two uses a custom-developed CNN to classify detected regions of interest/signs from stage one.

Stage one uses transfer learning. We used the model architecture provided by Ultralytics and tested it on a dummy dataset to ensure it worked before training it on our actual dataset. By default, YOLOv11m both identifies and classifies signs within an image. However, we chose to implement only the identification layer to address a bottleneck identified by a course TA during our project proposal review, as the proposed traditional computer vision approach would be the project's main failpoint due to inevitable inconsistencies. During training, the YOLO model processes the resized images and outputs bounding boxes for detected signs. At the end of training, the model provides the best performing weights based on validation metrics, which are then used for inference. Each region of interest is cropped and resized to 224x224 pixels for classification.

Stage two uses a custom convolutional neural network developed by the team, implemented in PyTorch. The network consists of four main convolution blocks, each containing two 3x3 convolution layers with a stride of one, a padding of one, and no bias, followed by batch normalization and a ReLU activation function. The first convolution block starts with 32 filters and doubles subsequently to 64, 128, and 256 filters. After each block, there is also 2x2 max pooling to reduce spatial dimensions, followed by global average pooling, and dropout with a probability of 0.2, and a fully connected layer that maps 256 hidden units with ReLU activation. The final fully connected layer outputs logits corresponding to the 399 classes being evaluated, which are then passed to a softmax in the loss function. This architecture cleanly separates identification and classification, using YOLOv11m's transfer learning for detection, and the specialized CNN for fine-grained classification, enabling high detection accuracy while still maintaining computational efficiency.

6 BASELINE MODEL

In order to compare our model to establish a minimalistic baseline, we implemented a model inspired by the gesture recognition exercise from Lab 3. Its baseline is a shallow CNN with only two convolutional layers, followed by max-pooling, and two fully connected hidden layers. This architecture has very few trainable parameters when compared to deeper networks, allowing for faster training and reduced computational requirements.

As discussed previously, input images were resized to 224x224 pixels and normalized. The model's feature extraction relied solely on the two convolutional layers, which means it could capture only basic spatial patterns such as edges and simple shapes. Hence, it lacked the depth and capacity to model the complex variations in lighting, weather, occlusions, or finer differences between visually similar traffic signs.

Training was performed directly on the cropped traffic sign images from our dataset, using the class labels provided in the CSV files that we created for our dataset in stage 2. Cross-entropy loss was minimized using the Adam optimizer, and performance was evaluated in terms of classification accuracy on a validation set.

While this baseline CNN is significantly less complicated than our final model, it serves an important role as a sanity check, allowing us to verify that our dataset, preprocessing pipeline, and labelling are functional before investing in more computationally complex architectures. Comparing against this baseline also quantifies the performance gains provided by a deeper CNN.

7 QUANTITATIVE RESULTS

Over the course of 60 epochs, the Stage one classifier steadily improved, with validation accuracy starting at around 65% and climbing to just under 88% by the end. Training accuracy followed a similar upward trend, eventually nearing 99%, while both training and validation loss decreased in parallel. A small gap emerged between the two losses toward the later epochs, suggesting that validation performance had started to plateau with signs of overfitting. These figures are interpolated from training logs and plots rather than taken from exact recorded values, while the model was trained on a transfer learning architecture.

For Stage 2, the detection model was trained for 100 epochs and reached an estimated precision of about 0.78, with a recall of around 0.65. The mean average precision at 50% IoU (mAP@50) rose to roughly 0.71, and the stricter mAP@50–95 averaged about 0.53. Across training, box loss, classification loss, and distribution focal loss all decreased for both training and validation sets. The recall–confidence curve started at around 82% at very low confidence thresholds and decreased as the threshold increased.

When integrated into a complete pipeline, the system achieved an end-to-end accuracy of roughly 72% on the held-out test set, where a correct prediction required both detecting and correctly classifying the sign. The primary bottleneck was Stage 2’s recall. Most errors came from missed detections rather than misclassifications of detected signs, meaning that improving detection sensitivity without sacrificing precision would likely lead to the largest gains in overall performance.

8 QUALITATIVE RESULTS

When our data signs were clear and cropped, the classifier was extremely accurate, even for signs like similar speed limits styles. If the picture was sharp and nothing blocked the sign, it was confident in the prediction. With faded paint, partial blockages, or awkward angles, it got most accurate, but confidence dropped. This matches the validation accuracy, as it was great in good conditions, but less certain when the input quality was poor. The detection stage was solid for large, centered signs, with boxes that lined up on the edges. It worked fine in cloudy or lightly shaded lighting, but missed more signs when they were small or hidden. Several false positives popped up throughout testing, usually from bright-colored objects that resembled traffic signs. When we ran the full pipeline, it did well with one or two obvious signs in the frame, but performance dipped if there were lots of signs close together or small in the image. Most mistakes came from missing detections, not misclassifying detected signs, which is why the classifier’s accuracy is higher than the full pipeline’s 72%. Overall, it’s reliable when conditions are clean, but smaller, blocked, or blurry signs throw it off. The fastest way to improve the system would be to make detection better for those tricky cases.



Figure 3: Same image classified differently in different conditions

As can be seen in this image above, the conditions are dark and the image is blurry, the classifier incorrectly classifies the yield on right turn sign as the “regulatory” class. The images above are all examples of our model working on never tested before data that we personally collected for testing purposes. Compared to this image below which identifies the exact same sign correctly in a more clear and sharper image.

9 EVALUATE MODEL ON NEW DATA

To test our model and ensure it works in practice, we've collected data from different angles, times of day, camera qualities, and quantities of signs. We took 23 photos during the day, night, in the suburbs, downtown, with a professional camera, an old phone, lots of signs in the picture, and with no signs in the picture. Testing edge cases will ensure our model performs well in different scenarios which the model can be deployed. The images we used to test our model were not used in the training of hyperparameters and were collected exclusively for evaluation. We preprocessed the new images the same way as the training data and ran them through the full pipeline of YOLO detection first, and then the CNN classification on each detected sign. On this set, the detector produced confident predictions for large, clearly visible signs, with detection confidence often in the 0.8–0.92 range and classification confidence above 0.95. When conditions were less ideal, small signs, partial occlusion, extreme angles, or poor lighting, detection scores sometimes fell below 0.5, and a few signs were missed entirely. False positives also appeared when non-sign objects had similar colors or shapes, such as construction markers or bright advertisements. In total, for this new dataset, the detector consistently identified the major, centrally located signs but showed a drop in recall compared to the original test set. Classification accuracy for detected signs remained high, but the end-to-end accuracy was lower, mainly because of missed detections. These results give us a realistic look at how the system works outside of the training/test sets. The model performs well in good conditions and produces high-confidence, correct predictions, but detection recall is the main bottleneck. Improving sensitivity without hurting precision would be the most effective next step. Since these images were collected specifically for evaluation and have never been used for training or tuning, they provide a benchmark of what performance would look like if it were applied on the streets.

10 DISCUSSION

Overall, the model performed well based on the results collected during testing for both stage one and two. The model achieved a test accuracy of 83% for stage one, while it increased to 87% for stage two, compared to the baseline 79%.

These testing accuracies prove that it was the correct decision to use transfer learning, as opposed to using traditional computer vision as initially suggested in our project proposal. This improvement allowed our system to shift dependancy onto the second classification stage, always giving it a fair chance to perform.

Something unusual regarding stage 2 results was that training accuracy neared 97% while validation plateaued at 87%. While both accuracies plateaued after the 20th epoch, this large difference was noteworthy, but an overfitting result of a poorly balanced dataset, as class 88 dominated the stage 2 data. Due to this imbalance in data, our model had less time training on rare classes, leading to poor generalization on “other-signs” when tested on those same classes, leading to the drop off in accuracy.

Despite these constraints, validation performance stayed the same over the remaining 80 epochs, suggesting our CNN architecture was designed well. However, that said, we learned that it is extremely important to have balanced class datasets, as one class dominating the dataset can lead to the model blindly predicting a certain class more often than others.

11 ETHICAL CONSIDERATIONS

Our traffic sign recognition system is designed to make roads safer, but like many machine learning projects, it carries certain ethical challenges that need to be considered before it's ever deployed outside of a test environment.

Dataset bias poses one of the greatest risks, with large datasets like MTSD tending to overrepresent common signs from certain countries. This means the model could perform well during testing but fail to recognize an unusual sign in a different part of the world, which can have serious consequences in a real driving scenario. Another consequence of our MTSD dataset is having an unbalanced number of images from each class, resulting in a more accurate model for certain types of signs but not for the majority of other classes.

The limitations of the model itself also matter. Our approach works best with clear, high-quality images. Factors like poor lighting, rain, glare, or a partially hidden part of the sign can reduce accuracy. Even a single incorrect reading of something as important as a stop sign or speed limit could lead to unsafe driving decisions. That risk puts a strong ethical responsibility on anyone deploying the system to ensure it has been tested under many different real-world conditions.

There are also privacy concerns when collecting additional training data. Capturing street scenes often means recording people, vehicles, or license plates without consent, which brings both ethical and legal implications.

Some steps can help reduce these risks:

- Expanding the training set with diverse images from different regions, lighting conditions, and weather scenarios..
- Stress-testing the system with edge cases and unusual examples.
- Using interpretability tools (e.g., Grad-CAM) to check how the model reaches its decisions.
- Blurring or removing any identifiable features in collected images.
- Being transparent about what the model can and cannot do, and keeping a human in the loop for decision-making in critical environments.

By actively considering these issues and building safeguards into the process, it's possible to reduce harm, improve fairness, and make the system more trustworthy for real-world use.

REFERENCES

- N.S. Artamonov and P.Y. Yakimov. Towards real-time traffic sign recognition via yolo on a mobile gpu, 2018. , Last accessed on 2025-06-12.
- Dan Cireşan. Multi-column deep neural networks for image classification, 2012. , Last accessed on 2025-06-12.
- Christian Ertler. Mapillary traffic sign dataset, 2019. , Last accessed on 2025-07-10.
- Google Maps. Google maps - street view, 2025. , Last accessed on 2025-06-12.
- Susano Mingwin. Revolutionizing traffic sign recognition: Unveiling the potential of vision transformers, 2024. , Last accessed on 2025-06-12.
- Mykola. Gtsrb - german traffic sign recognition benchmark, 2018. , Last accessed on 2025-06-12.
- Atiqur Rahman. Fused-net: Detecting traffic signs with limited data, 2025. , Last accessed on 2025-07-10.
- Chi-Yi Tsai. Real-time embedded implementation of robust speed-limit sign recognition using a novel centroid-to-contour description method, 2017. , Last accessed on 2025-06-12.
- Ultralytics. Explore ultralytics yolov8, 2023. , Last accessed on 2025-06-12.